

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

(Университет ИТМО)

Факультет Прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

**ОТЧЕТ ПО Лабораторной работе 6
«Работа с БД в СУБД MongoDB»**

Обучающийся: Ипатова Ульяна Юрьевна К3239

Санкт-Петербург 2026

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

СУБД MongoDB. Вставка данных. Выборка данных

Практическое задание 2.1.1 - вставка единорогов

Задание: создать базу данных learn, заполнить коллекцию unicorns, добавить документ вторым способом и проверить содержимое коллекции.

```
use learn
```

```
db.unicorns.insertMany([{'name': 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63}, {'name': 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43}, {'name': 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182}, {'name': 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99}, {'name': 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80}, {'name': 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40}, {'name': 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39}, {'name': 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2}, {'name': 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33}, {'name': 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54}, {'name': 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}])
```

```

test> use learn
switched to db learn
learn> db.unicorns.drop()
true
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0dfad48549938235abc114'),
    '1': ObjectId('6a0dfad48549938235abc115'),
    '2': ObjectId('6a0dfad48549938235abc116'),
    '3': ObjectId('6a0dfad48549938235abc117'),
    '4': ObjectId('6a0dfad48549938235abc118'),
    '5': ObjectId('6a0dfad48549938235abc119'),
    '6': ObjectId('6a0dfad48549938235abc11a'),
    '7': ObjectId('6a0dfad48549938235abc11b'),
    '8': ObjectId('6a0dfad48549938235abc11c'),
    '9': ObjectId('6a0dfad48549938235abc11d'),
    '10': ObjectId('6a0dfad48549938235abc11e')
  }
}
learn> |

```

Затем документ был добавлен вторым способом (из методички): сначала объект был сохранен в переменную, после этого вставлен в коллекцию.

```
var document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

```
db.unicorns.insertOne(document)
```

```

learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a0dfb628549938235abc11f')
}
learn> |

```

Для проверки содержимого коллекции был выполнен запрос:

db.unicorns.find() - вывел содержимое коллекции.

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0dfad48549938235abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0dfad48549938235abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0dfad48549938235abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0dfad48549938235abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0dfad48549938235abc118'),
    name: 'Unicorn',
    loves: [ 'apple', 'carrot' ],
    weight: 1200,
    gender: 'm',
    vampires: 150
  }
]
```

```
{
  {
    _id: ObjectId('6a0dfad48549938235abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0dfad48549938235abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
}
```

```

{
  _id: ObjectId('6a0dfad48549938235abc11b'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a0dfad48549938235abc11c'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a0dfad48549938235abc11d'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a0dfad48549938235abc11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a0dfb628549938235abc11f'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}

```

Количество 12

```

learn> db.unicorns.countDocuments()
12
learn> |

```

Практическое задание 2.2.1

Задание: вывести списки самцов и самок единорогов, ограничить список самок первыми тремя особями, отсортировать списки по имени. Также найти самок, которые любят carrot, и ограничить выборку первой особью с помощью findOne и limit.

Вывод списка самцов единорогов, отсортированных по имени:

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0dfb628549938235abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0dfad48549938235abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    ...
  }
]
```

```
{
  {
    _id: ObjectId('6a0dfad48549938235abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0dfad48549938235abc117'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0dfad48549938235abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
}
```


Вывод списка самок с сортировкой по имени:

```
db.unicorns.find({gender: 'f'}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0dfad48549938235abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0dfad48549938235abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0dfad48549938235abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
```

Вывод первых трех самок после сортировки по имени:

```
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a0dfad48549938235abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0dfad48549938235abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

Поиск всех самок, которые любят carrot:

```
db.unicorns.find({gender: 'f', loves: 'carrot'})
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0dfad48549938235abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0dfad48549938235abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
learn> |
```

Ограничение результата первой найденной особью с помощью findOne:

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0dfad48549938235abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> |
```

Ограничение результата первой найденной особью с помощью limit:

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a0dfad48549938235abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |
```

Практическое задание 2.2.2

Задание: модифицировать запрос для вывода списка самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
```

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a0dfb628549938235abc11f'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a0dfad48549938235abc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a0dfad48549938235abc117'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0dfad48549938235abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
```

```
learn> |
```

Практическое задание 2.2.3

Задание: вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({$natural: -1})
```

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a0dfb628549938235abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0dfad48549938235abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
]
```

```

    {
      _id: ObjectId('6a0dfad48549938235abc11b'),
      name: 'Raleigh',
      loves: [ 'apple', 'sugar' ],
      weight: 421,
      gender: 'm',
      vampires: 2
    },
    {
      _id: ObjectId('6a0dfad48549938235abc11a'),
      name: 'Kenny',
      loves: [ 'grape', 'lemon' ],
      weight: 690,
      gender: 'm',
      vampires: 39
    },
    {
      _id: ObjectId('6a0dfad48549938235abc119'),
      name: 'Ayna',
      loves: [ 'strawberry', 'lemon' ],
      weight: 733,
      gender: 'f',
      vampires: 40
    },
    {
      _id: ObjectId('6a0dfad48549938235abc118'),
      name: 'Solnara',
      loves: [ 'apple', 'carrot', 'chocolate' ],
      weight: 550,
      gender: 'f',
      vampires: 80
    },
    {
      _id: ObjectId('6a0dfad48549938235abc117'),
      name: 'Rooooooodles',
      loves: [ 'apple' ],
      weight: 575,
      gender: 'm',
      vampires: 99
    }
  ],
  learn> |

```

```

    {
      _id: ObjectId('6a0dfad48549938235abc116'),
      name: 'Unicrom',
      loves: [ 'energon', 'redbull' ],
      weight: 984,
      gender: 'm',
      vampires: 182
    },
    {
      _id: ObjectId('6a0dfad48549938235abc115'),
      name: 'Aurora',
      loves: [ 'carrot', 'grape' ],
      weight: 450,
      gender: 'f',
      vampires: 43
    },
    {
      _id: ObjectId('6a0dfad48549938235abc114'),
      name: 'Horny',
      loves: [ 'carrot', 'papaya' ],
      weight: 600,
      gender: 'm',
      vampires: 63
    }
  ]
  learn> |

```

Практическое задание 2.1.4

Задание: вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```

```
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}}).pretty()
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> |
```

Логические операторы

Практическое задание 2.3.1

Задание: вывести список самок единорогов весом от полутонны до 700 кг, исключив `_id`.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0}).sort({name: 1})
[
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
learn> |
```


Практическое задание 2.3.2

Задание: вывести список самцов весом от полутонны и предпочитающих грапе и lemon, исключив _id.

```
db.unicorns.find({gender: 'm',weight: {$gte: 500},loves: {$all: ['grape',  
'lemon']}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'm',weight: {$gte: 500},loves: {$all: [  
grape', 'lemon']}}, {_id: 0})  
[  
  {  
    name: 'Kenny',  
    loves: [ 'grape', 'lemon' ],  
    weight: 690,  
    gender: 'm',  
    vampires: 39  
  }  
]  
learn> |
```

Практическое задание 2.3.3

Задание: найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find({vampires: {$exists: false}})
```

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0dfad48549938235abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Практическое задание 2.3.4

Задание: вывести упорядоченный список имён самцов единорогов с информацией об их первом предпочтении.

```
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> |
```

Запрос к вложенным объектам

Практическое задание 3.1.1

Задание: создать коллекцию городов с вложенным объектом mayor, найти города с независимыми мэрами и города с беспартийными мэрами.

```
db.towns.insertMany([
  {name: 'Punxsutawney', population: 6200, last_census:
    ISODate('2008-01-31'), famous_for: ['Phil the groundhog'], mayor: {name: 'Jim
    Wehrle'}},
  {name: 'New York', population: 22200000, last_census: ISODate('2009-
    07-31'), famous_for: ['Statue of Liberty', 'food'], mayor: {name: 'Michael
    Bloomberg', party: 'I'}},
  {name: 'Portland', population: 528000, last_census:
    ISODate('2009-07-20'), famous_for: ['beer', 'food'], mayor: {name: 'Sam
    Adams', party: 'D'}}])
```

```
db.towns.find()
```

```
learn> db.towns.insertMany([
  {name: "Punxsutawney", populatiuon: 6200, last_
    last_sensus: ISODate("2008-01-31"), famous_for: [""], mayor: { name: "Jim W
    ehrle"}},
  {name: "New York", populatiuon: 22200000, last_sensus: ISODate("20
    09-07-31"), famous_for: ["status of liberty", "food"], mayor: {name: "Micha
    el Bloomberg", party: "I"}},
  { name: "Portland", populatiuon: 528000, last_se
    last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {n
    name: "Sam Adams", party: "D"} }])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0e06bc8549938235abc120'),
    '1': ObjectId('6a0e06bc8549938235abc121'),
    '2': ObjectId('6a0e06bc8549938235abc122')
  }
}
learn> |
```

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0e06bc8549938235abc120'),
    name: 'Punxsutawney',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a0e06bc8549938235abc121'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0e06bc8549938235abc122'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> |
```

Найти города с независимыми мэрами

В методичке независимый мэр (party: "I")

```
db.towns.find({'mayor.party': 'I'})
```

```
learn> db.towns.find({"mayor.party": "I"},{_id: 0, name: 1, mayor: 1})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> |
```

Найти города с беспартийными мэрами. Беспартийный мэр — это мэр, у которого поле party отсутствует.

```
db.towns.find({'mayor.party': {$exists: false}})
```

```
learn> db.towns.find({"mayor.party": {$exists: false}},{_id: 0, name: 1,
mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

JavaScript и курсоры

Практическое задание 3.1.2

Задание: создать функцию для вывода самцов, создать курсор для первых двух самцов, отсортировать их по имени и вывести через forEach.

Функция о выводе информации о единороге:

```
function printUnicorn(unicorn) {print(unicorn.name + '-' + unicorn.weight + ' кг')}
```

```
learn> function maleUnicorns() {return db.unicorns.find({gender: "m"},{_id: 0, name: 1, gender: 1})}
[Function: maleUnicorns]
learn> maleUnicorns()
[
  { name: 'Horny', gender: 'm' },
  { name: 'Unicrom', gender: 'm' },
  { name: 'Roooooodles', gender: 'm' },
  { name: 'Kenny', gender: 'm' },
  { name: 'Raleigh', gender: 'm' },
  { name: 'Pilot', gender: 'm' },
  { name: 'Dunx', gender: 'm' }
]
learn> |
```

Затем был создан курсор, который выбирает самцов, сортирует их по имени и ограничивает результат двумя документами.

```
var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)
```

После этого курсор был обработан с помощью метода forEach.

```
learn> var cursor = db.unicorns.find({gender: "m"},{_id: 0, name: 1, gender: 1}).sort({name: 1}).limit(2)
learn> cursor.forEach(function(obj) {printjson(obj)})
{
  name: 'Dunx',
  gender: 'm'
}
{
  name: 'Horny',
  gender: 'm'
}
learn> |
```

Агрегированные запросы

Практическое задание 3.2.1

Нужно вывести количество самок весом от полутонны до 600 кг.

```
db.unicorns.countDocuments({gender: 'f',weight: {$gte: 500, $lte: 600}})
```

```
learn> db.unicorns.countDocuments({gender: "f",weight: {$gte: 500, $lte:  
600}})  
2  
learn> |
```

Практическое задание 3.2.2

Нужно вывести список предпочтений единорогов.

```
db.unicorns.distinct('loves')
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> |
```


Практическое задание 3.2.3

Нужно посчитать количество единорогов каждого пола.

```
db.unicorns.aggregate([{$group: {_id: '$gender', count: {$sum: 1}}}])
```

```
learn> db.unicorns.aggregate([{$group: { _id: "$gender", count: {$sum: 1}
}}}])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn> |
```

Редактирование данных

Практическое задание 3.3.1

Задание: добавить единорога Barny.

```
db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
```

```
learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a0e09f78549938235abc123')
}
learn> db.unicorns.find( {name: "Barny"}, {_id: 0})
[ { name: 'Barny', loves: [ 'grape' ], weight: 340, gender: 'm' } ]
learn> |
```

```
learn> db.unicorns.countDocuments()
13
learn> |
```

Практическое задание 3.3.2

Задание: изменить данные Ауна: вес теперь 800, количество убитых вампиров - 51.

```
db.unicorns.updateOne({name: 'Ayna'},{$set: {weight: 800,vampires: 51}})
```

```
learn> db.unicorns.updateOne({name: "Ayna"},{$set: {weight: 800,vampires:
51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Ayna"},{_id: 0, name: 1, weight: 1, vampi
vampires: 1})
[ { name: 'Ayna', weight: 800, vampires: 51 } ]
learn> |
```

Практическое задание 3.3.4

Задание: увеличить всем самцам количество убитых вампиров на 5.

```
db.unicorns.updateMany({gender: 'm'},{$inc: {vampires: 5}})
```

```
learn> db.unicorns.updateMany( {gender: "m"},{$inc: {vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: "m"},{_id: 0, name: 1, gender: 1, vampir
vampires: 1}).sort({name: 1})
[
  { name: 'Barney', gender: 'm', vampires: 5 },
  { name: 'Dunx', gender: 'm', vampires: 170 },
  { name: 'Horny', gender: 'm', vampires: 68 },
  { name: 'Kenny', gender: 'm', vampires: 44 },
  { name: 'Pilot', gender: 'm', vampires: 59 },
  { name: 'Raleigh', gender: 'm', vampires: 7 },
  { name: 'Rooooooodles', gender: 'm', vampires: 104 },
  { name: 'Unicrom', gender: 'm', vampires: 187 }
]
```

Практическое задание 3.3.5

Нужно удалить у Pilot предпочтение chocolate.

```
db.unicorns.updateOne({name: 'Pilot'},{$pull: {loves: 'chocolate'}})
```

```
learn> db.unicorns.updateOne({name: "Pilot"},{$pull: {loves: "chocolate"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Pilot"},{_id: 0, name: 1, loves: 1})
[ { name: 'Pilot', loves: [ 'apple', 'watermelon' ] } ]
learn> |
```

Практическое задание 3.3.6

Задание: добавить предпочтения: Aurora любит sugar, Rooooooodles любит lemon.

Команда для Aurora:

```
db.unicorns.updateOne({name: 'Aurora'},{$addToSet: {loves: 'sugar'}})
```

```
learn> db.unicorns.updateOne( {name: "Aurora"},{$addToSet: {loves: "sugar"
}}}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> |
```

Команда для Rooooooodles:

```
db.unicorns.updateOne({name: 'Rooooooodles'},{$addToSet: {loves: 'lemon'}})
```

```
learn> db.unicorns.updateOne({name: "Rooooooodles"},{$addToSet: {loves: "l
emon"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> |
```

Проверка:

```
learn> db.unicorns.find({name: {$in: ["Aurora", "Rooooooodles"]}}, {_id: 0,
name: 1, loves: 1})
[
  { name: 'Aurora', loves: [ 'carrot', 'grape', 'sugar' ] },
  { name: 'Rooooooodles', loves: [ 'apple', 'lemon' ] }
]
learn> |
```

Практическое задание 3.3.7

Нужно удалить ключ `vampires` у всех самок.

```
db.unicorns.updateMany({gender: 'f'},{$unset: {vampires: ''}})
```

```
learn> db.unicorns.updateMany({gender: "f"},{$unset: {vampires: ""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 4,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: "f"},{_id: 0, name: 1, gender: 1, vampir
vampires: 1}).sort({name: 1})
[
  { name: 'Aurora', gender: 'f' },
  { name: 'Ayna', gender: 'f' },
  { name: 'Leia', gender: 'f' },
  { name: 'Nimue', gender: 'f' },
  { name: 'Solnara', gender: 'f' }
]
learn> |
```

Практическое задание 3.4.1

Нужно удалить документы с беспартийными мэрами.

Кто будет удалён

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1,
mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

db.towns.deleteMany({'mayor.party': {\$exists: false}})

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0e06bc8549938235abc121'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0e06bc8549938235abc122'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> |
```


Практическое задание 3.4.2

Нужно очистить коллекцию towns.

```
db.towns.deleteMany({})
```

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()

learn> db.towns.countDocuments()
0
learn> |
```

Практическое задание 3.4.3

Нужно вывести список доступных коллекций.

```
learn> show collections
towns
unicorns
learn> |
```

Ссылки в БД

В MongoDB нет жёстких связей как в SQL через foreign key.

Практическое задание 4.1.1

Нужно создать коллекцию зон обитания единорогов и добавить ссылки на зоны в несколько документов единорогов.

```
db.habitats.insertMany([{_id: 'forest',name: 'Лесная зона',description: 'Зона обитания с большим количеством деревьев и укрытий'},{_id: 'meadow',name: 'Луговая зона',description: 'Открытая травянистая территория'},{_id: 'mountain',name: 'Горная зона',description: 'Высокогорная территория'}])
```

```
db.habitats.find()
```

```
learn> db.habitats.insertMany([{_id: "forest",name: "Magic Forest",description: "Forest zone with trees, berries and lakes"},{_id: "mountains",name: "Northern Mountains",description: "Cold mountain zone with rocks and snow"},{_id: "meadow",name: "Sunny Meadow",description: "Open grassland zone with flowers and rivers"}])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'meadow' }
}
learn> db.habitats.find()
[
  {
    _id: 'forest',
    name: 'Magic Forest',
    description: 'Forest zone with trees, berries and lakes'
  },
  {
    _id: 'mountains',
    name: 'Northern Mountains',
    description: 'Cold mountain zone with rocks and snow'
  },
  {
    _id: 'meadow',
    name: 'Sunny Meadow',
    description: 'Open grassland zone with flowers and rivers'
  }
]
learn> |
```

Добавление ссылок нескольким единорогам

Лесная зона:

```
db.unicorns.updateMany({name: {$in: ['Horny', 'Solnara']}},{ $set: {habitat: {$ref: 'habitats', $id: 'forest'}}})
```

```
learn> db.unicorns.updateMany({name: {$in: ["Aurora", "Solnara", "Nimue"]}},{$set: {habitat: {$ref: "habitats", $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

Луговая зона:

```
db.unicorns.updateMany({name: {$in: ['Aurora', 'Rooooooodles']}},{$set: {habitat: {$ref: 'habitats', $id: 'meadow'}}})
```

```
learn> db.unicorns.updateMany({name: {$in: ["Horny", "Pilot", "Dunx"]}},{$set: {habitat: {$ref: "habitats", $id: "meadow"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
learn> |
```

Горная зона:

```
db.unicorns.updateOne({name: 'Unicrom'},{$set: {habitat: {$ref: 'habitats', $id: 'mountain'}}})
```

```
learn> db.unicorns.updateMany({name: {$in: ["Unicrom", "Rooooooodles"]}},{$set: {habitat: {$ref: "habitats", $id: "mountains"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
learn> |
```

Проверка содержимого unicorns

```
db.unicorns.find({habitat: {$exists: true}},{_id: 0, name: 1, habitat: 1})
```

```
learn> db.unicorns.find({habitat: {$exists: true}},{_id: 0, name: 1, habitat: 1})
[
  { name: 'Horny', habitat: DBRef('habitats', 'meadow') },
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountains') },
  { name: 'Rooooooodles', habitat: DBRef('habitats', 'mountains') },
  { name: 'Solnara', habitat: DBRef('habitats', 'forest') },
  { name: 'Pilot', habitat: DBRef('habitats', 'meadow') },
  { name: 'Nimue', habitat: DBRef('habitats', 'forest') },
  { name: 'Dunx', habitat: DBRef('habitats', 'meadow') }
]
learn> |
```

Проверка, что ссылка работает

```
learn> db[aurora.habitat.collection].findOne({_id: aurora.habitat.oid})
{
  _id: 'forest',
  name: 'Magic Forest',
  description: 'Forest zone with trees, berries and lakes'
}
learn> |
```

Настройка индексов

Практическое задание 4.2.1

Задание: проверить, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.

```
db.unicorns.createIndex({name: 1},{unique: true})
```

```
learn> db.unicorns.createIndex(
|   {name: 1},
|   {unique: true}
| )
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

Добавляем документ с уже существующим именем:

```
db.unicorns.insertOne({name: 'Aurora', loves: ['apple'], weight: 500, gender: 'f'})
```

```
learn> db.unicorns.insertOne({name: 'Aurora', loves: ['apple'], weight: 500, gender: 'f'})
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
learn> |
```

MongoDB выдает ошибку дублирования ключа.

Создаю пару индексов, чтобы в следующем задании было что удалять:

```
learn> db.unicorns.createIndex({name: 1})
name_1
learn> db.unicorns.createIndex({gender: 1})
gender_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1' },
  { v: 2, key: { gender: 1 }, name: 'gender_1' }
]
learn> |
```

Практическое задание 4.3.1

Получаю информацию обо всех индексах unicorns

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1' },
  { v: 2, key: { gender: 1 }, name: 'gender_1' }
]
learn> |
```

Удалить все индексы, кроме _id

```
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 3,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> |
```

```
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> |
```

Попытаться удалить индекс _id

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

План выполнения запроса

Практическое задание 4.4.1

Задание: создать коллекцию numbers на 100000 документов, выбрать последние 4 документа, посмотреть план запроса без индекса, создать индекс по value, снова посмотреть план запроса и сравнить время.

```
for (i = 0; i < 100000; i++) {db.numbers.insertOne({value: i})}
```

```
learn> for(i = 0; i < 100000; i++){db.numbers.insertOne({value: i})}
{
  acknowledged: true,
  insertedId: ObjectId('6a0e14fc8549938235ad47c3')
}
learn> db.numbers.countDocuments()
100000
learn> |
```

```
db.numbers.find({}, {_id: 0, value: 1}).sort({value: -1}).limit(4)
```

```
learn> db.numbers.find({}, {_id: 0, value: 1}).sort({value: -1}).limit(4)
[
  { value: 99999 },
  { value: 99998 },
  { value: 99997 },
  { value: 99996 }
]
learn> |
```

план запроса без индекса

```
var noIndexPlan = db.numbers.find({}, {_id: 0, value: 1}).sort({value: -1}).limit(4).explain('executionStats')
```

noIndexPlan.executionStats.executionTimeMillis – время выполнения

noIndexPlan.executionStats.totalDocsExamined – количество просмотренных документов

noIndexPlan.queryPlanner.winningPlan – вывод плана выполнения


```

learn> var noIndexPlan = db.numbers.find({}, {_id: 0, value: 1}).sort({value: -1}).limit(4).explain("executionStats")

learn> noIndexPlan.executionStats.executionTimeMillis
248
learn> noIndexPlan.executionStats.totalDocsExamined
100000
learn> noIndexPlan.queryPlanner.winningPlan
{
  isCached: false,
  stage: 'PROJECTION_SIMPLE',
  transformBy: { _id: 0, value: 1 },
  inputStage: {
    stage: 'SORT',
    sortPattern: { value: -1 },
    memLimit: 104857600,
    limitAmount: 4,
    type: 'simple',
    inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
  }
}
learn> |

```

индекс для ключа value

```
db.numbers.createIndex({value: -1})
```

```

learn> db.numbers.createIndex({value: -1})
value_-1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: -1 }, name: 'value_-1' }
]
learn> |

```

```

learn> db.numbers.find({}, {_id: 0, value: 1}).sort({value: -1}).limit(4)
[
  { value: 99999 },
  { value: 99998 },
  { value: 99997 },
  { value: 99996 }
]
learn> |

```

```

learn> var withIndexPlan = db.numbers.find({}, {_id: 0, value: 1}).sort({v
value: -1}).limit(4).explain("executionStats")

learn> withIndexPlan.executionStats.executionTimeMillis
0
learn> withIndexPlan.executionStats.totalDocsExamined
0
learn> withIndexPlan.queryPlanner.winningPlan
{
  isCached: false,
  stage: 'LIMIT',
  limitAmount: 4,
  inputStage: {
    stage: 'PROJECTION_COVERED',
    transformBy: { _id: 0, value: 1 },
    inputStage: {
      stage: 'IXSCAN',
      nss: 'learn.numbers',
      keyPattern: { value: -1 },
      indexName: 'value_-1',
      isMultiKey: false,
      multiKeyPaths: { value: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { value: [ '[MaxKey, MinKey]' ] }
    }
  }
}
learn> |

```

```

learn> withIndexPlan.executionStats.nReturned
4
learn> |

```

после создания индекса: после создания индекса MongoDB использует индексный обход IXSCAN. В плане запроса может отображаться PROJECTION_COVERED, потому что запрос выводит только поле value, которое уже есть в индексе. Поэтому totalDocsExamined может быть равен 0, так как MongoDB получает нужные данные из индекса и не читает сами документы коллекции.

Вывод:

В ходе лабораторной работы была создана база данных `leapn` и коллекция `unicorns`. Были выполнены основные CRUD-операции: вставка, выборка, изменение и удаление данных. Также были рассмотрены запросы к массивам, вложенным объектам, использование логических операторов, сортировка, ограничение выборки, работа с курсорами и агрегированными запросами.

Дополнительно были изучены ссылки между коллекциями на примере зон обитания единорогов, а также настройка и удаление индексов. На примере коллекции `numbers` было показано, что индекс ускоряет выполнение запроса и меняет план выполнения с полного просмотра коллекции на использование индексного обхода.